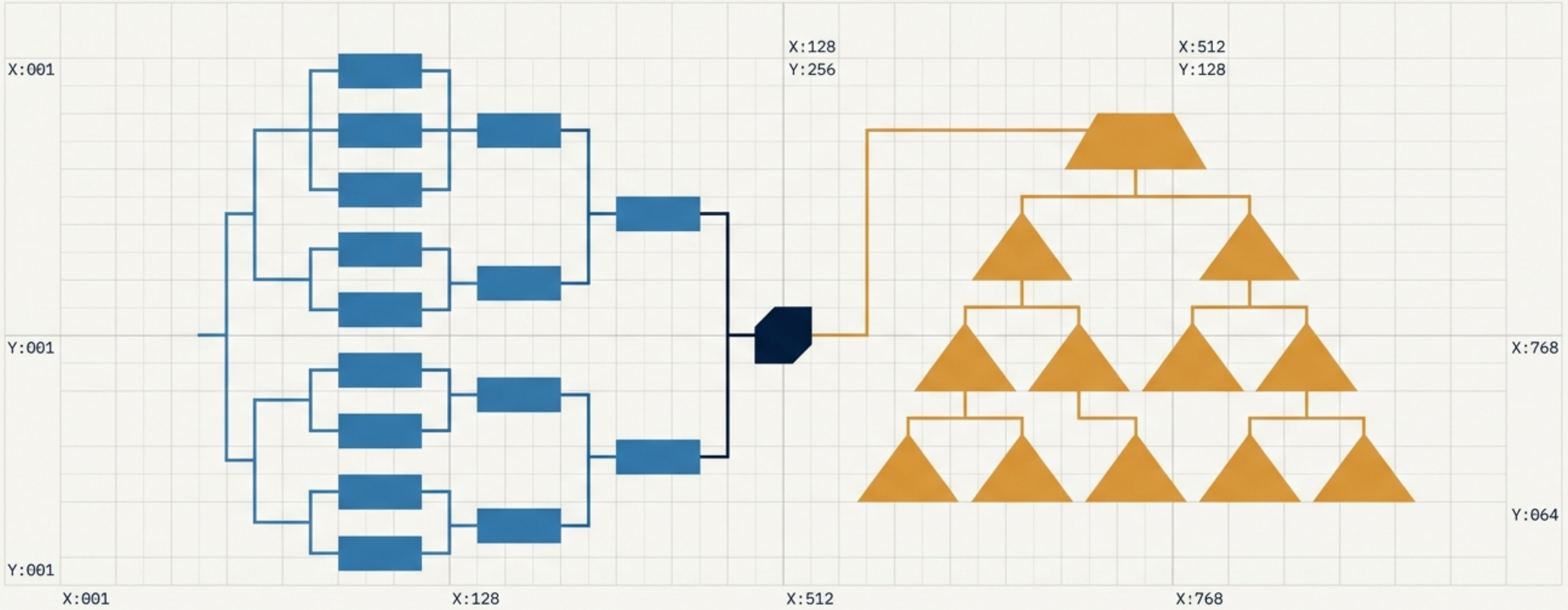


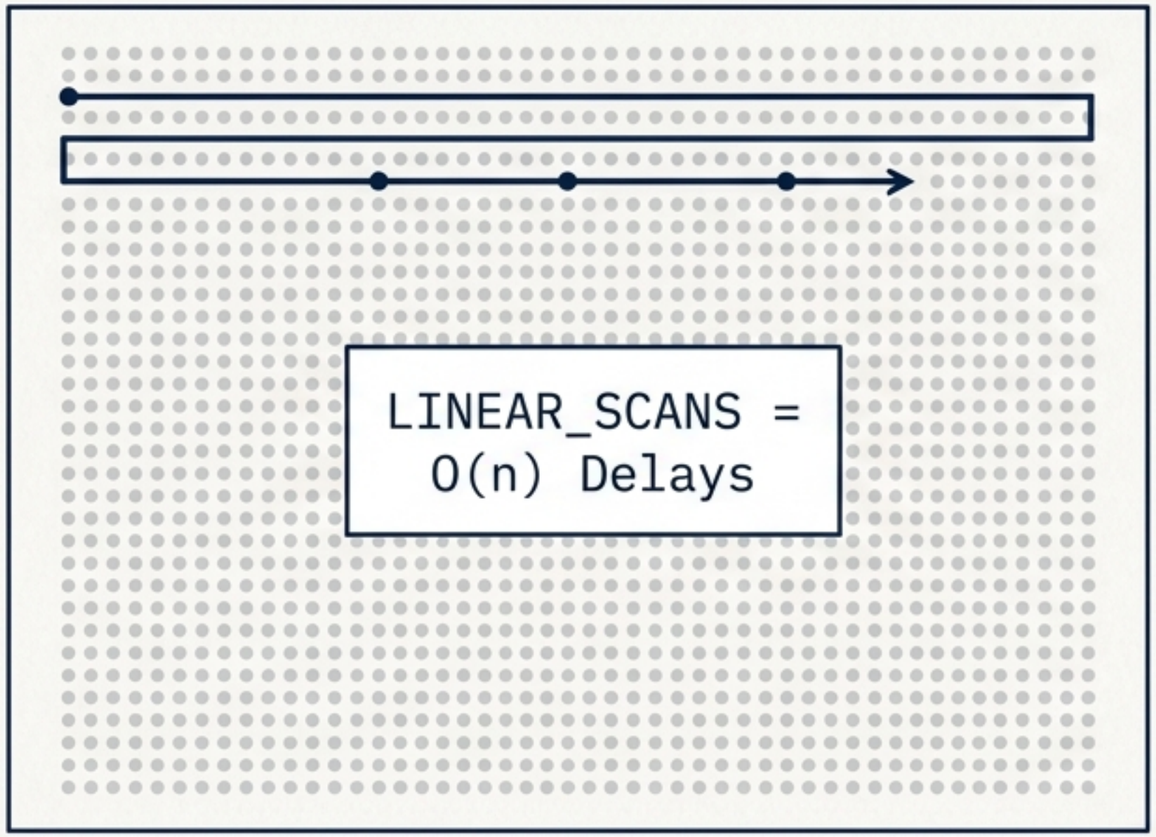
Smart Library Inventory & Priority Tracker

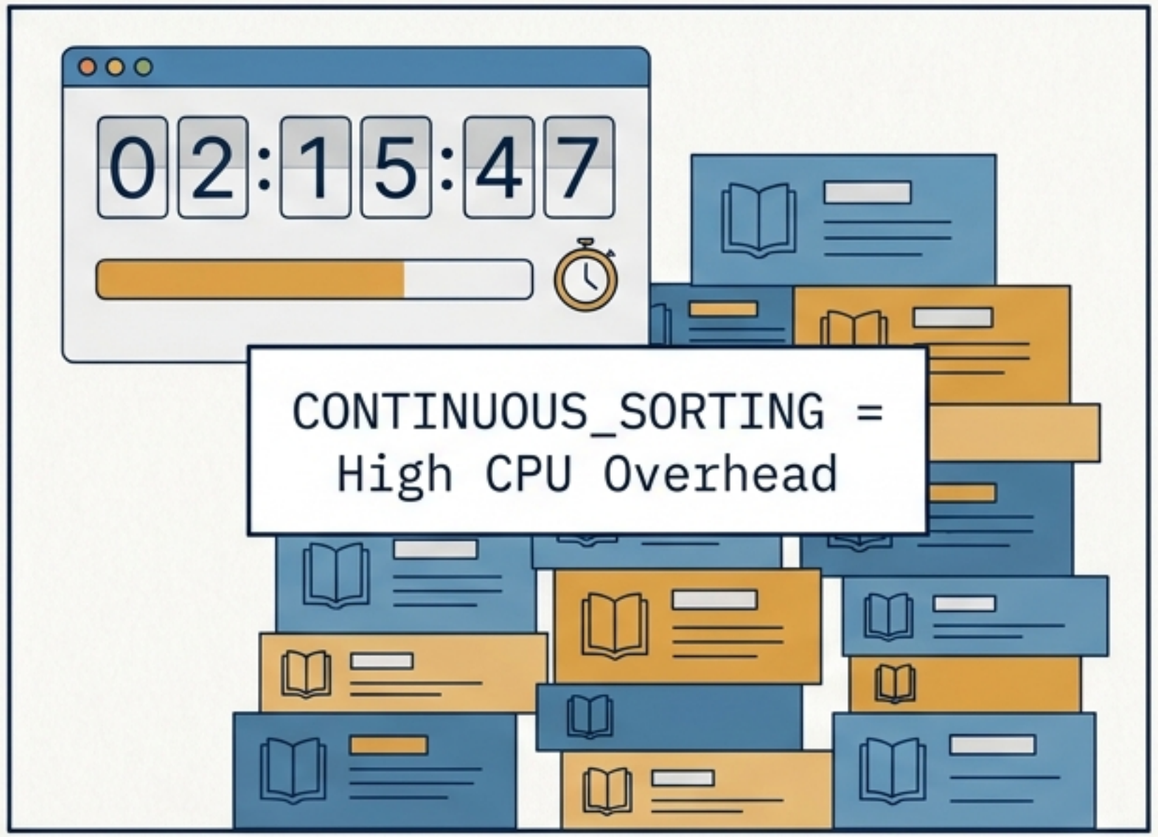
A high-performance, dual-structure architecture in C designed for O(log *n*) cataloging and O(1) due-date monitoring.

Gemini Notebook

Traditional library databases collapse when massive scale and time-sensitivity collide.

As library inventories scale, unoptimized database structures suffer severe performance degradation. They rely on slow linear scans for lookups and require heavy, constant resorting to track moving deadlines in real-time.





Gemini Notebook

Existing architectures fail to balance catalog speed with real-time deadline monitoring.

Architecture	Catalog Search Speed	Real-Time Priority Tracking	In-Memory Efficiency
Flat-File / Sequential Arrays	✗ (O(n) latencies)	✗	Low
Standard Relational Databases (SQL)	✓	✗ (Requires Heavy Sort Executions)	Medium
Pure Hash Maps	✓ (O(1) average)	✗ (Zero Chronological Sorting Capability)	High
Proposed Dual-Structure	✓	✓	**High**

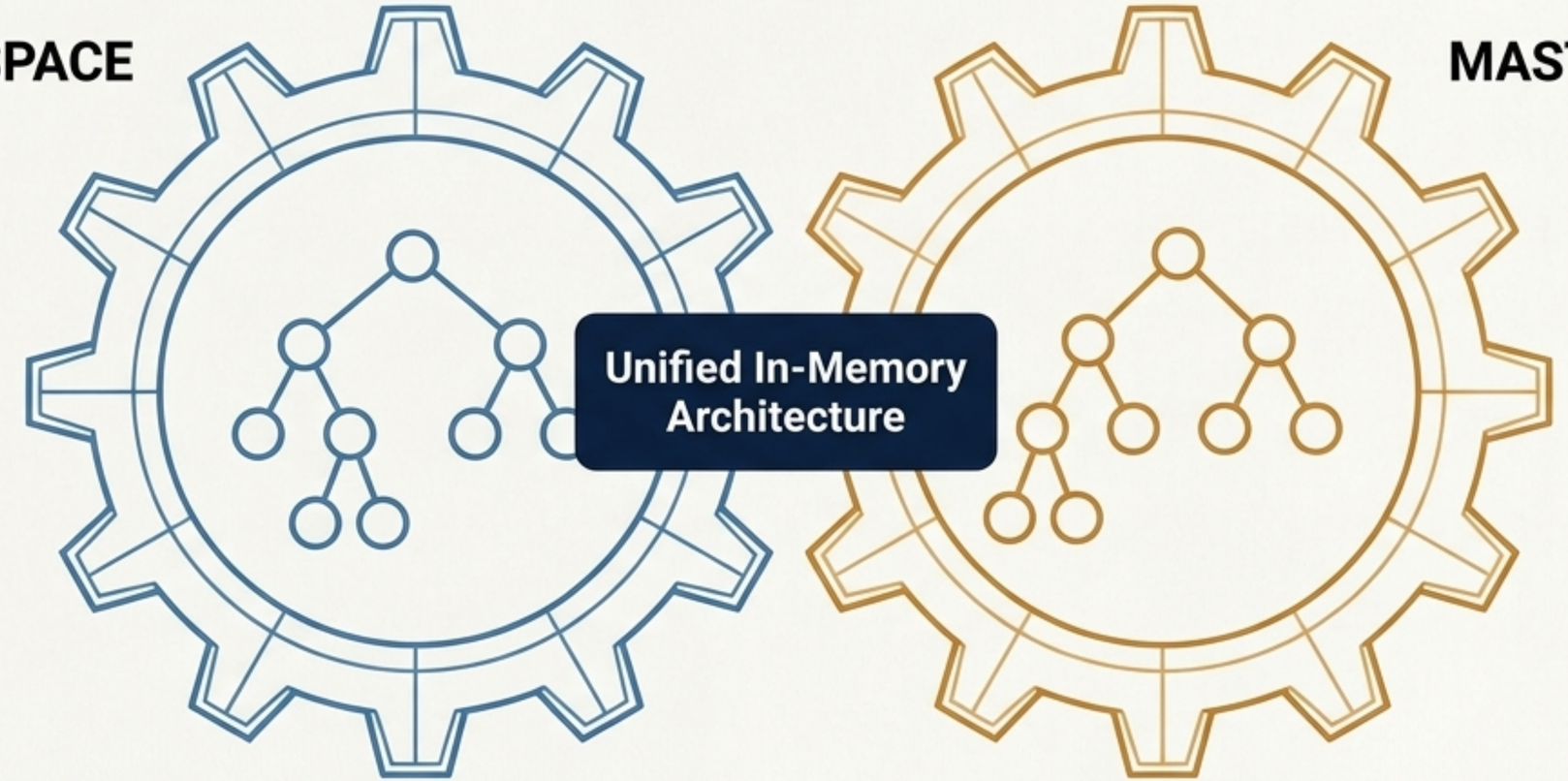
Existing solutions either fail to scale during inventory lookups or lack a unified architecture for automated overdue tracking.

Gemini Notebook

A dual-engine architecture decouples spatial organization from chronological sorting.

MASTERING SPACE

MASTERING TIME



The **Binary Search Tree** organizes massive catalogs for rapid hierarchical searching.

The **Min-Heap** manages moving deadlines, guaranteeing automated, time-sensitive overdue tracking without database resorting.

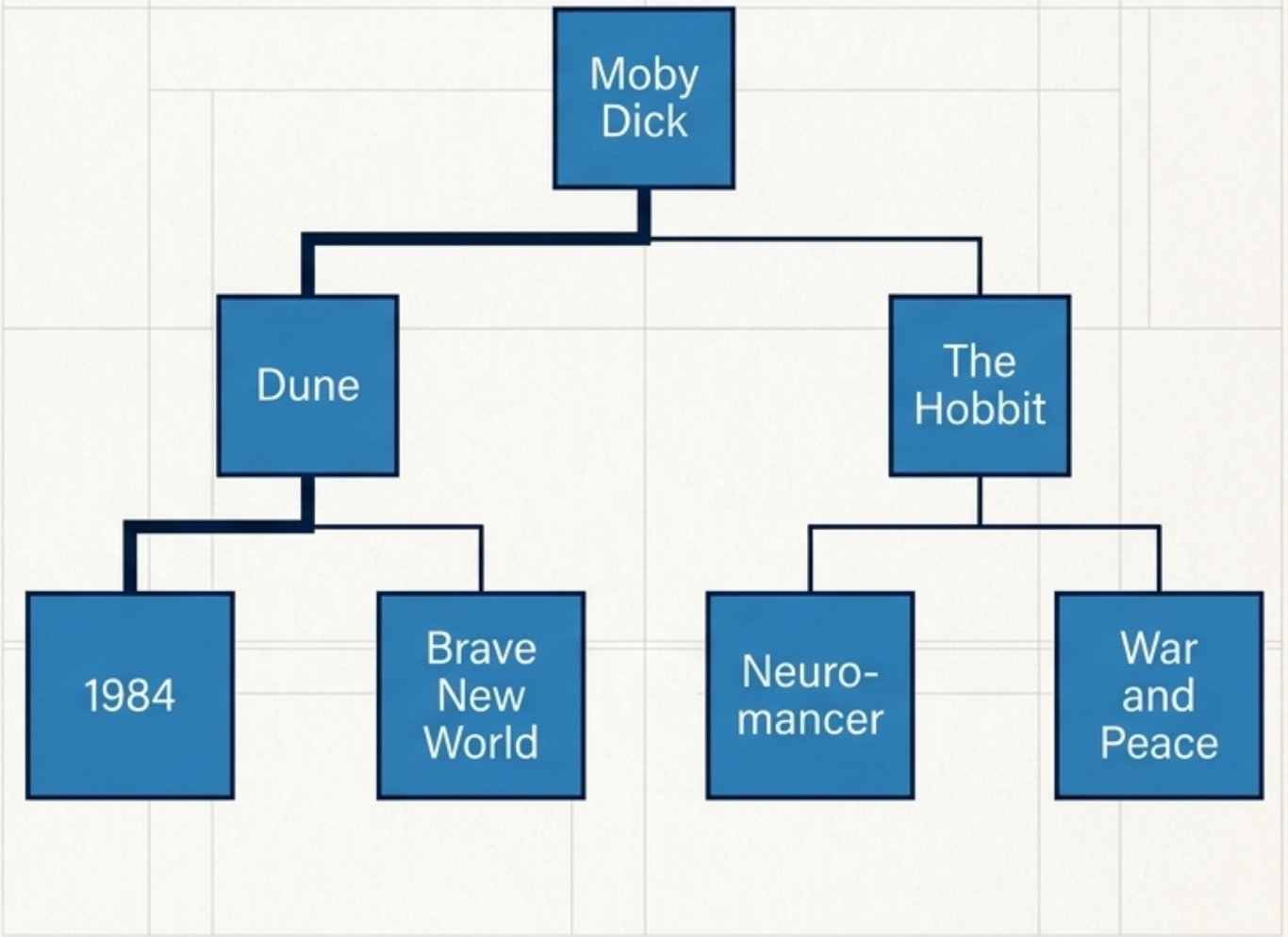
Gemini Notebook

The Binary Search Tree isolates target records in logarithmic time.

Structure:
In-Memory Binary Search Tree / Hash mapping.

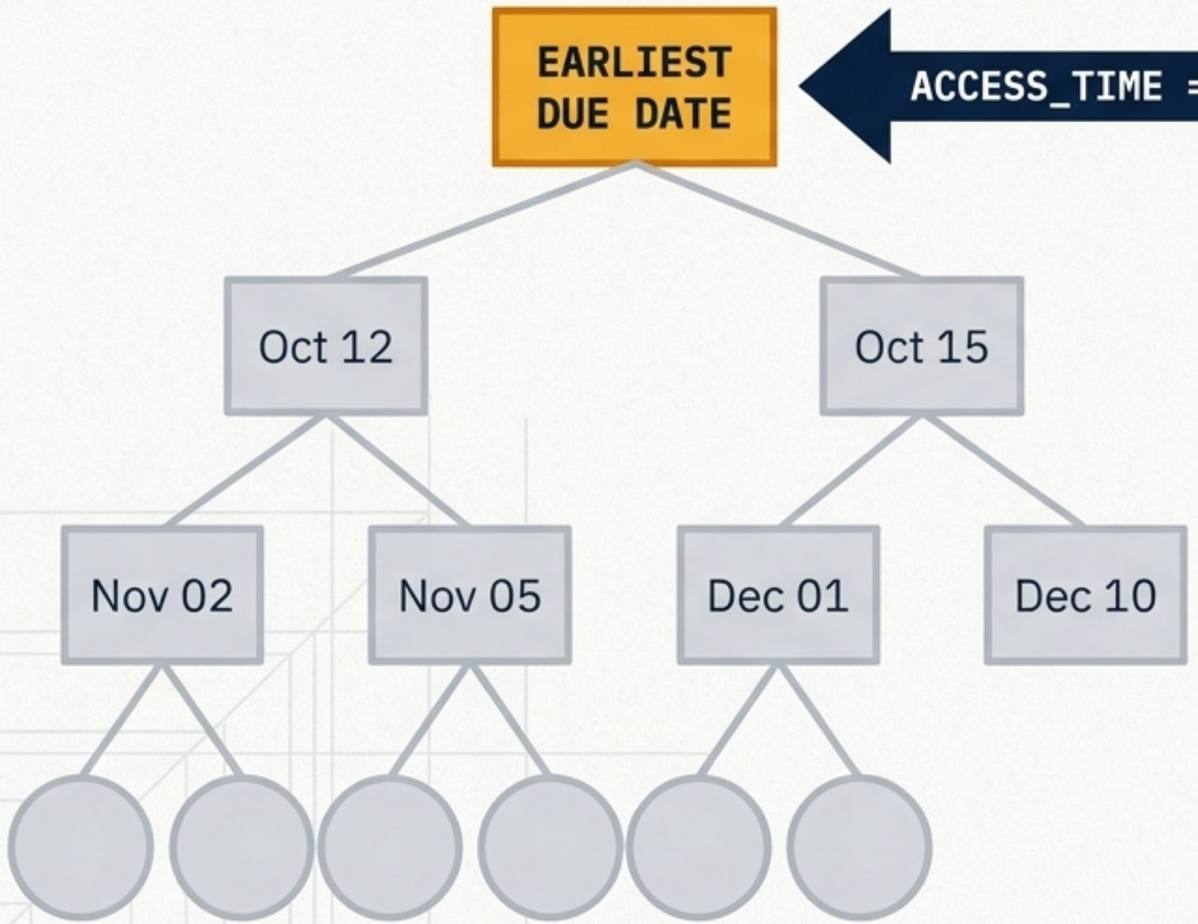
Operation:
Organizes the entire book inventory, routing queries recursively by title or author key.

Performance Metric:
Achieves guaranteed `O(log n)` time complexity for insertions and catalog lookups.



Gemini Notebook

A Min-Heap priority queue guarantees instant access to the most critical deadlines.



Structure: Priority Queue via Min-Heap.

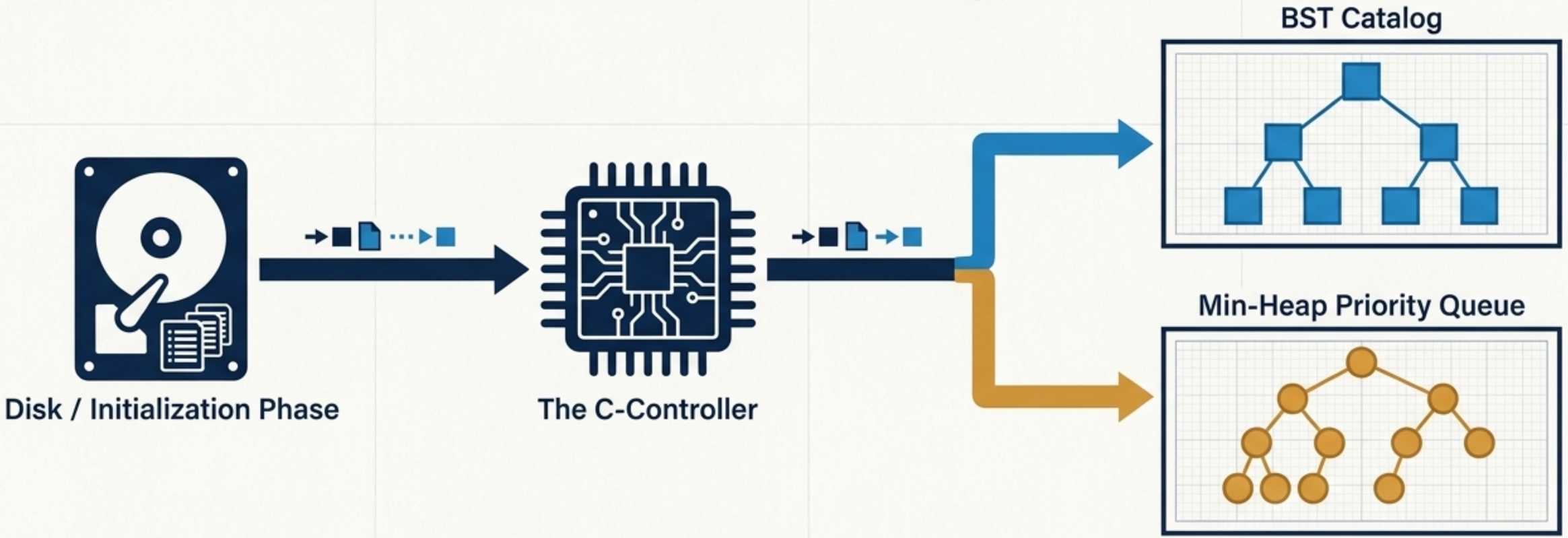
Operation: Manages book due dates chronologically immediately upon checkout.

Performance Metric: Keeps overdue or earliest deadlines constantly accessible at the root, entirely eliminating the need to execute linear sort operations on the database.

Gemini Notebook

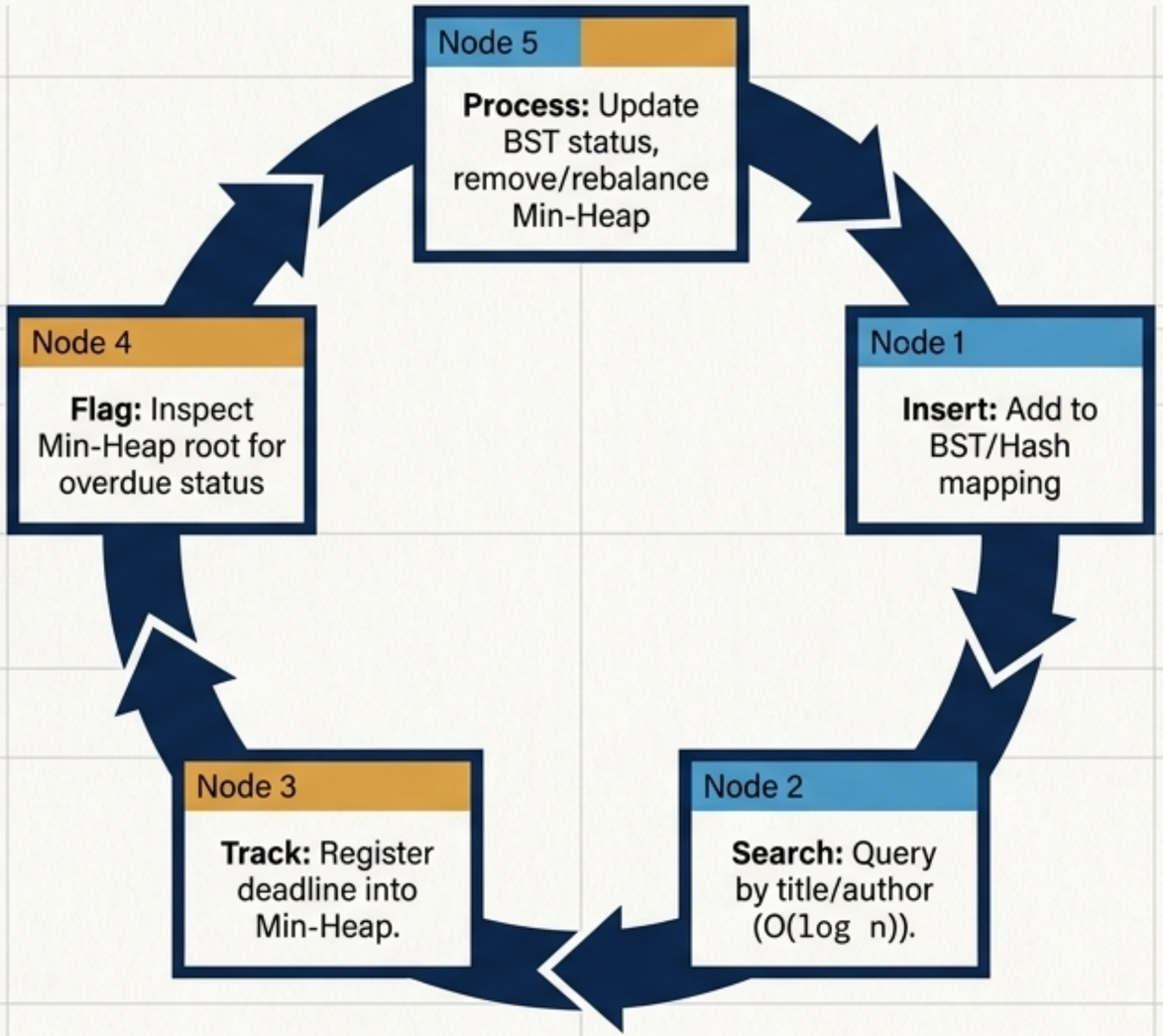
The central controller orchestrates a parallel data fork during memory initialization and execution.

Persistent records are loaded from disk into memory. User queries flow into the C controller, executing traversals on the BST for lookups, while simultaneously updating return nodes inside the Min-Heap during checkouts.



Gemini Notebook

A continuous closed-loop workflow governs the lifecycle of every catalog item.



Gemini Notebook

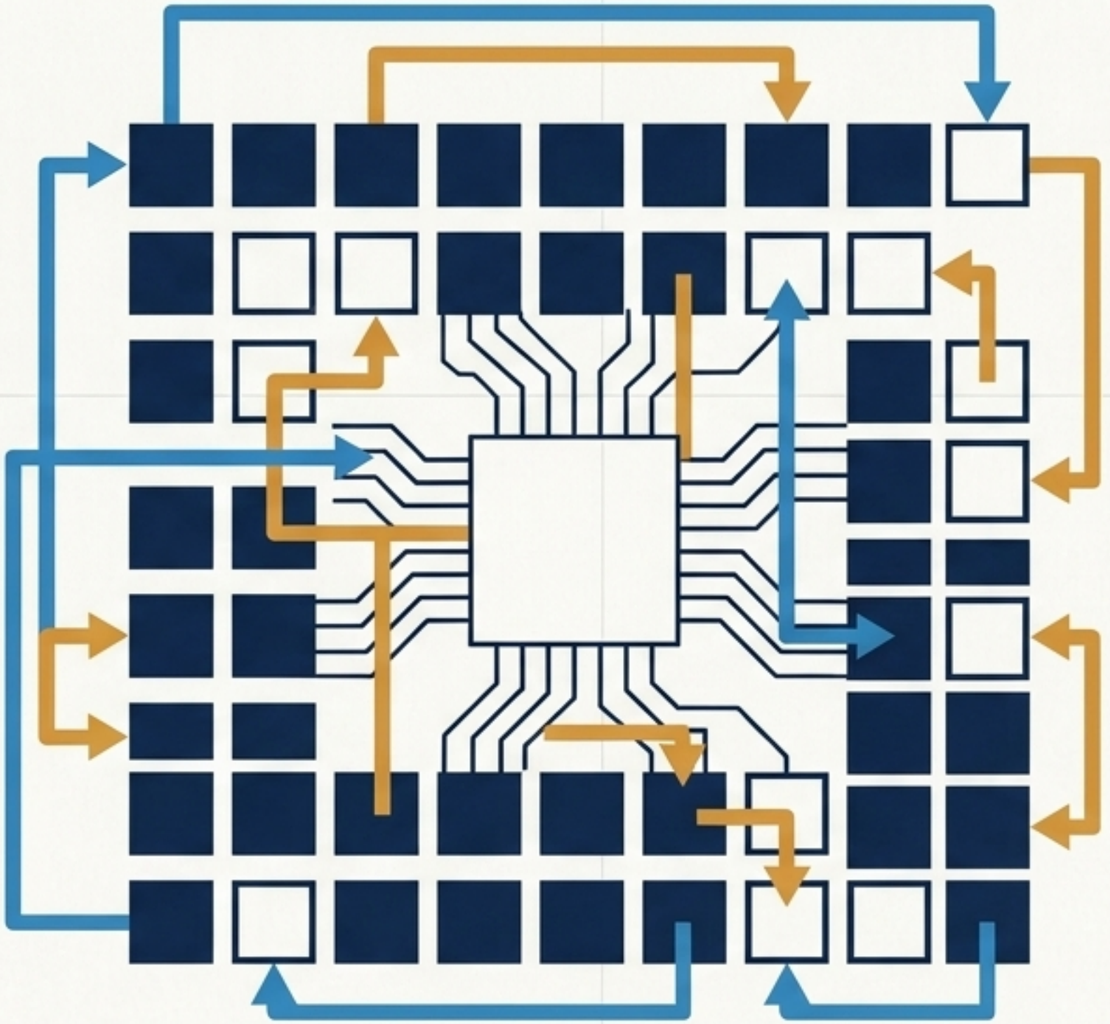
C provides the absolute memory and pointer control required for maximum execution efficiency.

Backend Architecture

C was selected for its precise memory management capabilities, allowing absolute pointer control over dynamic memory allocation for individual book nodes.

Frontend Interface

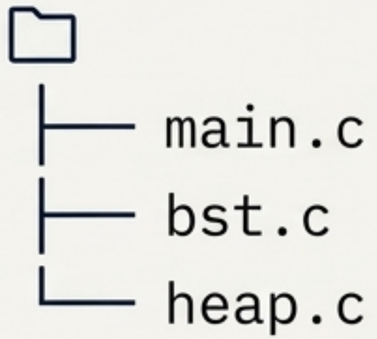
A clean C-based terminal/GUI interface (or embedded server bridge) presents searchable inventory tables and real-time overdue alerts without overhead bloat.



Gemini Notebook

Verified core modules confirm the mathematical efficiency of the dual-engine model.

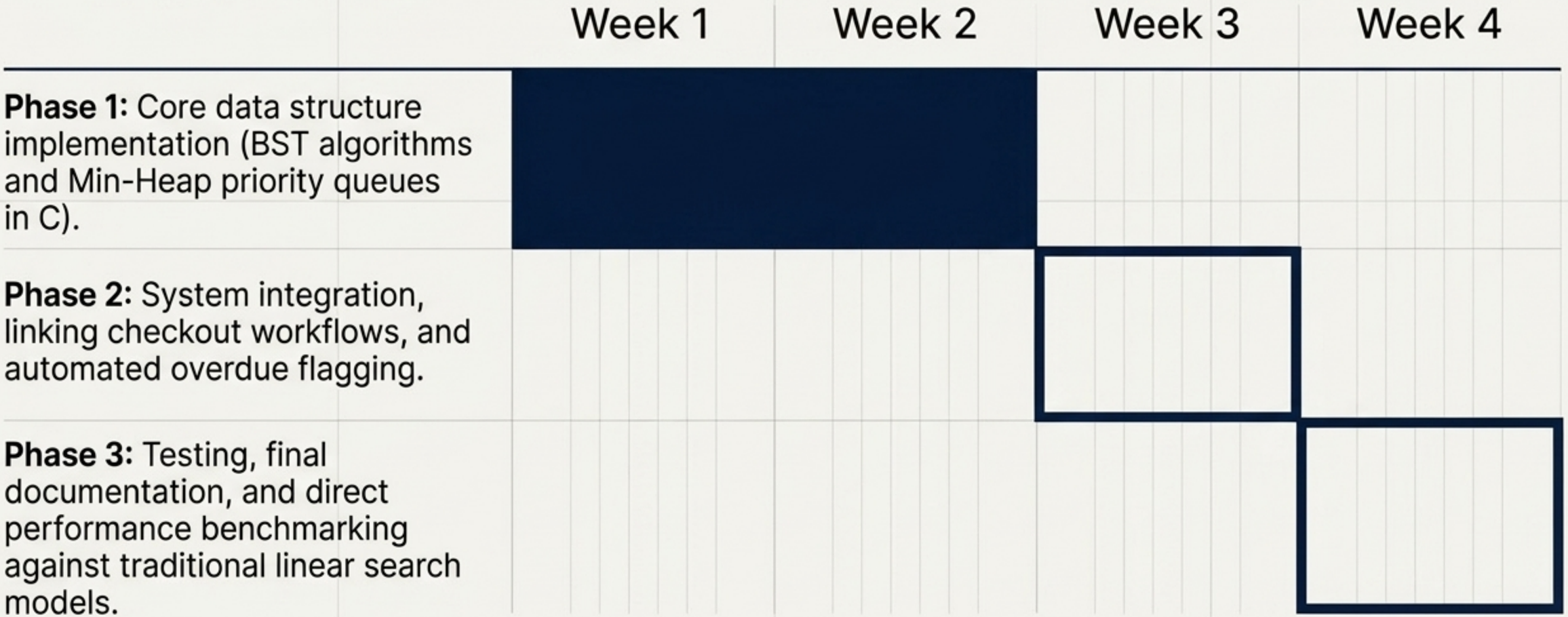
Foundational Modules



- Established dynamic memory allocation structures for individual book nodes.
- Completed BST insertion and search routines demonstrating verified `O(log n)` lookup speeds.
- Finalized Min-Heap insertion, deletion, and root-peeking algorithms for automated due-date monitoring.

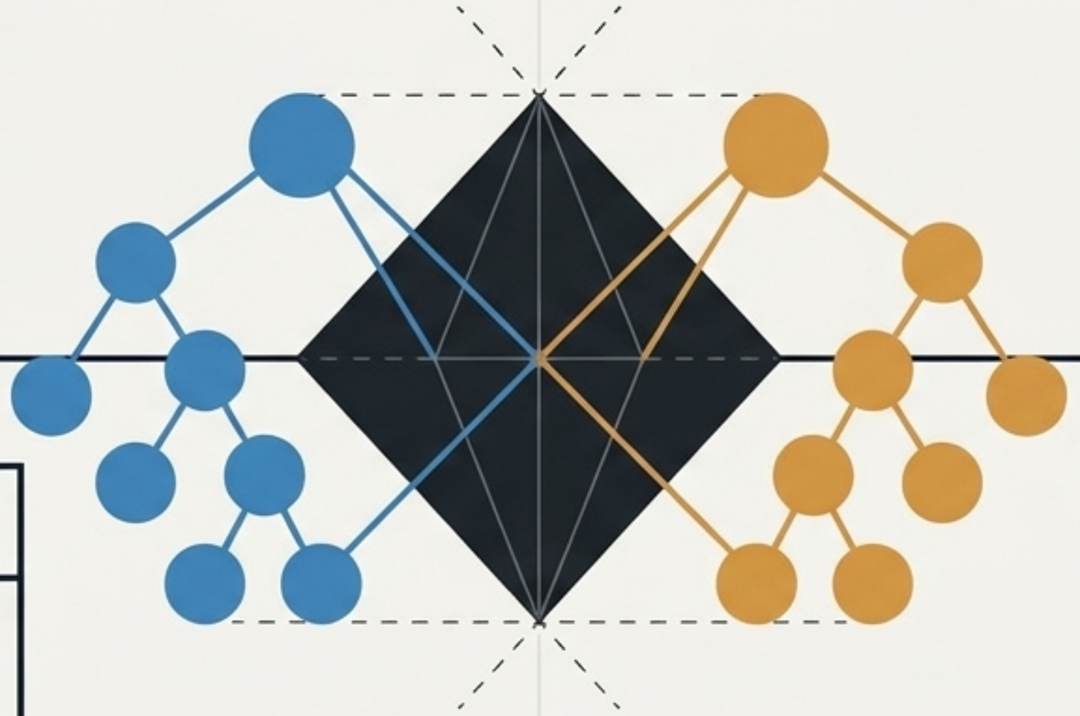
Gemini Notebook

A structured three-phase roadmap accelerates the transition from core algorithms to performance benchmarking.



Gemini Notebook

Optimized hierarchical data structuring eliminates search latency and guarantees infinite catalog scaling.



The Goal:

Definitively demonstrate performance superiority over traditional linear models through rigorous benchmarking.

The Result:

Zero latency on overdue tracking and instantaneous lookups, regardless of total library inventory volume.

Gemini Notebook